

COMP 60017: Systems Performance Engineering

Coursework 2 — Concurrency, parallelism and I/O

1 Synchronization primitives

We are implementing our own semaphore class, which is defined as follows:

```
class mysem {
public:
    mysem(uint32_t init_value);
    void acquire();
    void release();
private:
    std::atomic<uint32_t> counter;
};
```

The members of our semaphore class work as follows:

- `std::atomic<uint32_t> counter`: Internal counter (private member).
- `mysem::mysem(uint32_t init_value)`: Sets the initial value of the semaphore's internal counter.
- `mysem::acquire()`: Decreases the semaphore's internal counter by one if it's greater than 0, or otherwise waits until it is greater than zero.
- `mysem::release()`: Increases the semaphore's internal counter by one.

1.1 Question: Execution order (1) [5 marks]

Given the following program, write down all the possible orderings of what the program prints to the screen:

```
int main(int argc, char**argv)
{
    mysem s(2);
    std::thread t1([&]() {
        s.acquire();
        std::cout << 1; random_work(); std::cout << 1;
        s.release();
    });
    std::thread t2([&]() {
        s.acquire();
        std::cout << 2; random_work(); std::cout << 2;
        s.release();
    });
    t1.join(); t2.join();
    std::cout << std::endl;
}
```

1.2 Question: Execution order (2) [5 marks]

Now write all possible orderings if `mysem`'s internal counter is initialized to 1.

1.3 Question: User-level implementation [20 marks]

Write down the implementation of the three methods above, exclusively using the methods available in the `std::atomic` class of C++.

You can check <https://en.cppreference.com/w/cpp/atomic/atomic> for a list of methods and their documentation, largely seen at class. When it comes to memory ordering, use the most restrictive default used when no explicit arguments are provided (i.e., strong ordering with `std::memory_order_seq_cst`).

1.4 Question: Hybrid implementation [15 marks]

We now want to evolve our semaphore implementation into a hybrid one, using the Linux `futex` syscall to efficiently wait (see `man 2 futex` for the full documentation). Reimplement your `acquire` and `release` methods to use `futex` after trying for a fixed number of times (we will use 100 iterations; no other changes are allowed to `mysem`). You can assume it is OK to do the 100 iterations again every time a thread returns from its call to `futex`, and that the `release` method will always unconditionally call `futex`.

Note that when using `futex`, you can simply cast `std::atomic` into a regular numeric data type:

```
uint32_t* counter_ptr = reinterpret_cast<uint32_t*>(&counter);
```

1.5 Question: A better hybrid implementation [5 marks]

Unconditionally calling `futex` on `mysem::release` leads to unnecessary system call overheads. Briefly describe how you would eliminate clearly unnecessary calls to `futex` when you know for sure no other threads are waiting inside the kernel (i.e., are “inside” a call to `futex`).

2 Networked service design

We are writing a memcached-like server, a key/value store over the network (i.e., a hash table). This server has two operations: `GET(key)` returns the value for a given key, and `SET(key, value)` sets it. The internal data structures use lock-free algorithms such that each operation can execute independently.

2.1 Network modeling [15 marks]

We have the following average distribution of GET/SET operations and their sizes:

- *GET request size*: 1 Byte for the GET operation identifier, 512 Bytes for the key.
- *GET response size*: 1024 Bytes for the value.
- *SET request size*: 1 Byte for the SET operation identifier, 512 Bytes for the key, 1024 Bytes for the value.
- *SET response size*: 1 Byte to acknowledge the end of the SET operation.
- *Operation distribution/ratios*: 90% GETs, 10% SETs.

Assuming a full-duplex 10 Gbps network card, calculate the maximum throughput (operations per second) that we could process if we were able to saturate the network. Show your working.

Note that requests and responses have different sizes, but every request needs a response. Here we are also ignoring the network header overheads to keep calculations simpler.

2.2 CPU modeling [20 marks]

We have also measured the average time it takes to process each operation:

- *GET operation latency*: 200 μsec
- *SET operation latency*: 500 μsec

Assuming a more realistic throughput of 130000 req/sec and the processing times just presented, calculate how many CPUs we would need to fully serve the rate of incoming requests. Show your working.

2.3 Cutting corners [15 marks]

To shorten development time, we used separate read-write locks (`std::shared_mutex` in C++) to guard access to each bucket of the service's hash table (instead of our initially intended lock-free algorithm). Each GET operation keeps a shared lock of the bucket it accesses, whereas each SET operation takes an exclusive lock until the value has been replaced at the end of the operation.

We then perform various experiments that run for 5 min each, using 10000 uniformly random keys and values with the following parameters and observed results:

- 1000 req/sec, 100% GETs $\rightarrow$ average latency: 200 μsec
- 100000 req/sec, 100% GETs $\rightarrow$ average latency: 207 μsec
- 100000 req/sec, 50% GETs, 50% SETs $\rightarrow$ average latency: 475 μsec

For each of these experiments, briefly explain how their results would compare to those of our ideal lock-free implementation, and why.